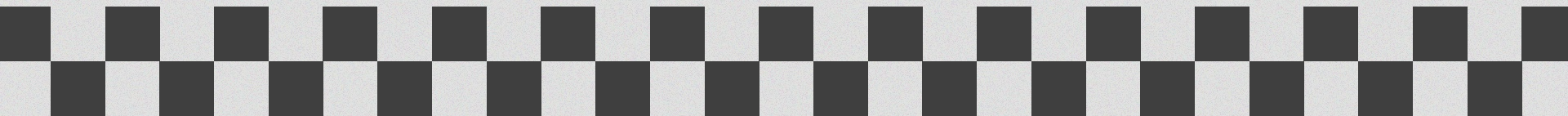


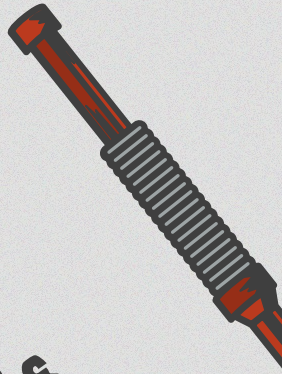
City Drift

Nancy Patel, Jennifer Martinez Mejia, Xavier
Thomas-Lewis, Jonas Schwab





Presentation Roadmap



01

Intro

02

Core Loop

03

Key Mechanics &
Demo

04

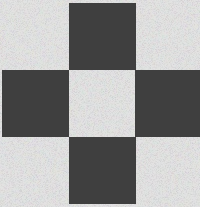
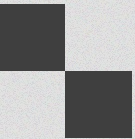
Feedback Loop &
Flow

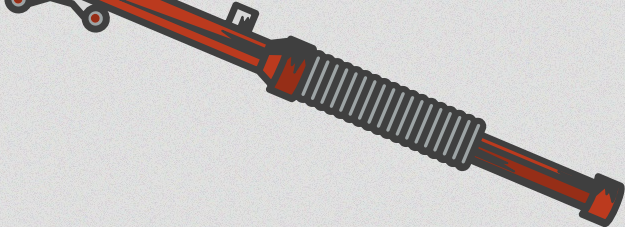
05

Collaborative Process
Postmortem

06

Development Process
Postmortem

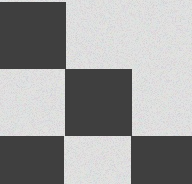




01

Intro

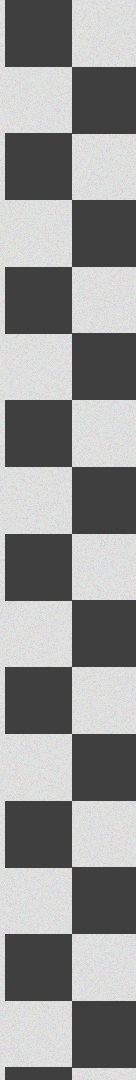
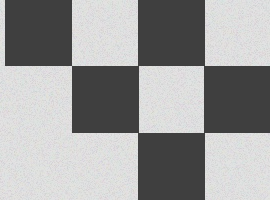
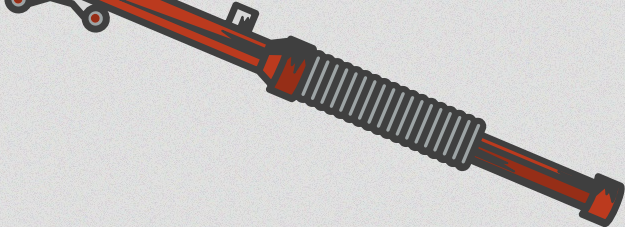
Our Team, Our Game



Our Game

Our game is “City Drift”, it is a multiplayer online game where players build their own race track to race against their friends. If they are playing alone they can race our AI agents. If they don’t have time to build, they can use a prebuilt track.





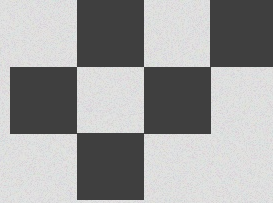
02

Core Loop

Input, Actions, Obstacle, Goal



Core Loop



Input

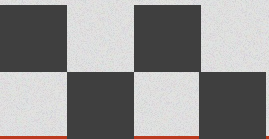
Racing & Building

We use mouse and keyboard inputs in order to control your car and to build your track, beyond simply navigating the game UI.

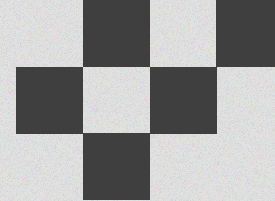
Actions

Add AIs & Invite Friends

Go through the tutorial to learn to play. Select Multi or Single Player. Add AI or real players to race against. Build your track to race. **Race.**



Core Loop



Obstacle

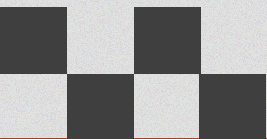
Add Obstacles, Time Limit, AIs, and Rubberbanding

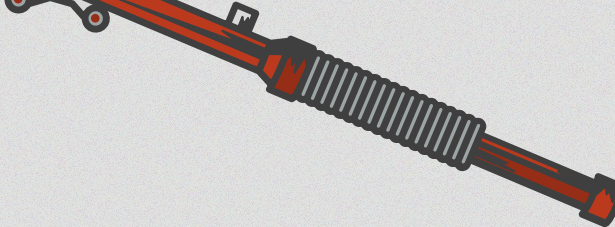
When building players can add obstacle track pieces that move & interfere. There is a time limit on map building to present a challenge. The racer in first place gives everyone who is a checkpoint behind a 10% speed boost to allow for them to catch up.

Goal

Win and Achieve High Scores

Beat the AI's or other players to get first place. Based on how fast you complete the track and the number of pieces in the track you will receive a score, that if it is high enough can place you on the global leaderboard.





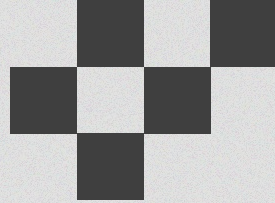
03

Key

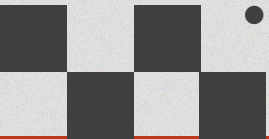
Mechanics

All the systems that make our game

Key **Mechanics**



- **Movement**
- **Track Systems (Track Builder & Prebuilt Tracks)**
- **Race Progression (Checkpoints & Countdown Start)**
- **Boost & Rubber-banding Systems**
- **AI Racers (Singleplayer Only)**
- **Race UI (Minimap & In-Game Chat)**
- **Multiplayer Lobby System**
- **Spectator Mode**
- **Leaderboard**



Key Mechanics

Movement:

The player controls their vehicle using standard directional inputs (WASD or Arrow Keys). Movement is physics-based, handling acceleration, turning, and collisions with the track and other racers.

Race Progression (Checkpoints & Countdown Start):

Each race begins with a countdown timer before players and AI are allowed to move. Checkpoints track racers' progress to

Spectator Mode

After finishing a race, players can enter spectator mode to watch remaining racers complete the track from different viewpoints.

Track Systems (Track Builder & Prebuilt Tracks):

Players can either race on one of three prebuilt tracks or create their own using the track builder. Custom tracks let the player control their experience and prebuilt tracks guarantee consistency and allow the player to race against AI racers.

Boost & Rubber-banding Systems:

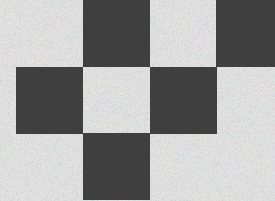
Players can activate a boost to temporarily increase speed. Rubber-banding helps balance races by giving players in the back a chance to catch up to the player in first.

Multiplayer Lobby System:

Players can join or create lobbies to play with others. Players can choose their name and color X button.

Leaderboard:

The leaderboard tracks and displays the highest scores across all players, encouraging competition and replayability.

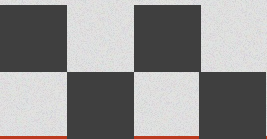


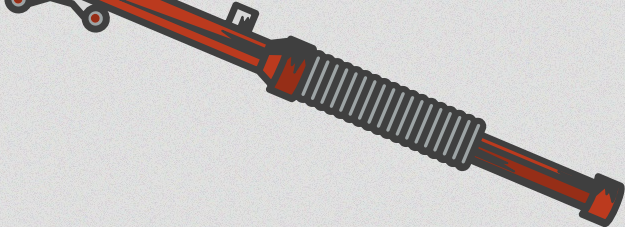
AI Racers (Singleplayer Only):

In singleplayer mode and on prebuilt tracks, AI-controlled racers compete against the player. These provide a challenge to anyone playing solo with varying difficulty levels.

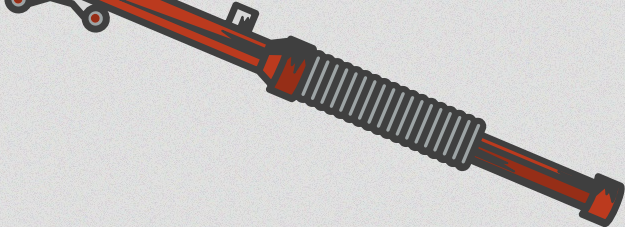
Race UI (Minimap & In-Game Chat):

The minimap displays the full track layout and racer positions in prebuilt tracks and shows the area around the player in player-built tracks. The chat system allows players to communicate with others during multiplayer races.





Demo



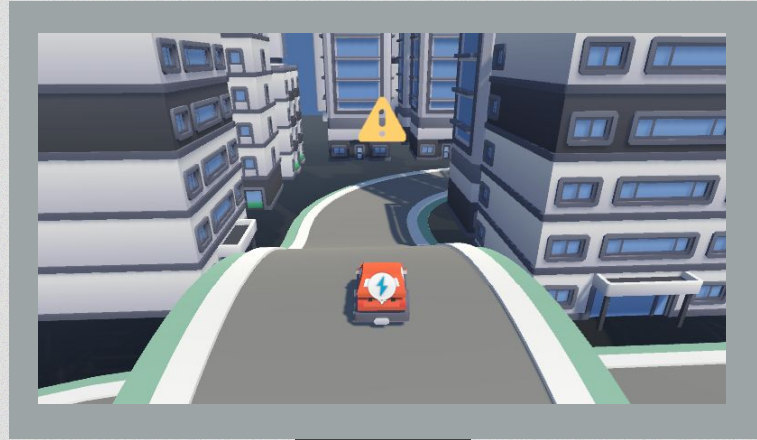
04

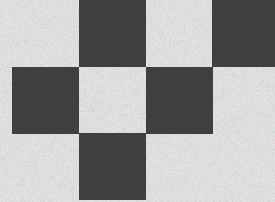
Feedback Loop & Flow

Feedback Loop:

Reversing

Each track has invisible directional checkpoints. A car moving backwards will cross the directional checkpoints in decreasing order. Once detected, a warning symbol will pop up (**trigger**). If the player does not start moving forwards (**action**), after a time delay, they will respawn at the closest crossed arch (**consequence**).

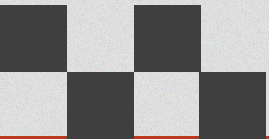
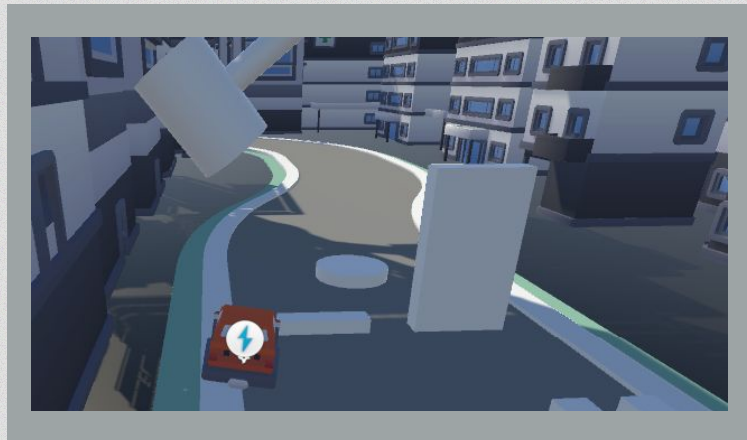


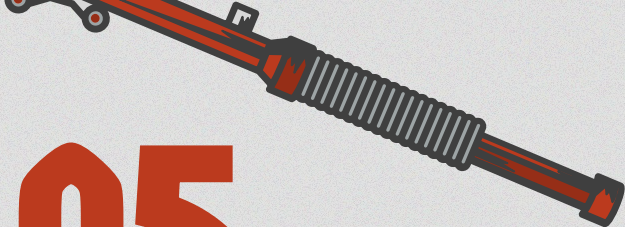


Flow:

Dynamic Obstacles

Currently, players/developers determine if and where obstacles go on the track. In the future, we could implement dynamic obstacles so that players who are leading in the game have more obstacles while players who are further behind have less obstacles to maneuver around.



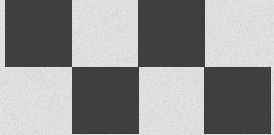


05

Collaborative Process

Postmortem

Pros, Cons, Suggestions

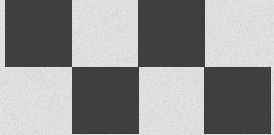


Collaborative Process Postmortem

Positives

Negatives

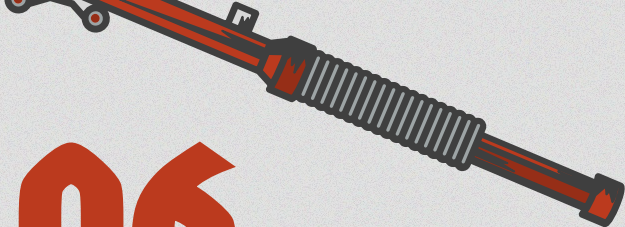
- Clear ownership where each person handled different parts of the game each week, and seamlessly updated them throughout.
 - Strong communication via texts and required attendance weekly check-ins.
 - We reflected on playtest results jointly to plan next steps.
 - Everyone contributed ideas, and we adjusted based on personal interests.
- Flexible team which made for easy brainstorming, but harder decision making
 - Dealt with a lot of merge conflicts due to Tuesday meetings forcing work-time overlap.
 - Meetings were hosted online, despite greater efficiency when working together before or after class.



Collaborative Process Postmortem

Effective Recommendations

- Use Github branches to avoid merge conflicts and therefore repetitive work.
- Set additional time to go beyond planning the core of the game, instead giving time to ideate the additional features that make it interesting [e.g. exploding cars].
- For larger components of the game, such as track building, don't pigeonhole yourself into a certain implementation, take time to assess that before even selecting assets [e.g. modular assets for cohesive building].

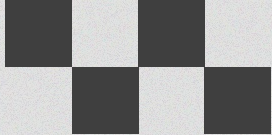


06

Development Process

Postmortem

Pros, Cons, Suggestions

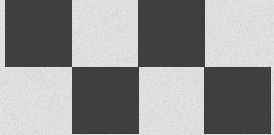


Development Process Postmortem

Positives

Negatives

- Began multiplayer very early on which helped continue building on it.
- Core systems like building and racing became solid through iteration.
- Used class concepts well in cameras, particle systems, and pathfinding AI.
- Picked assets from first week which helped our project look consistent.
- Spent a lot of time trying to figure out multiplayer on itch.io
- Had trouble finding a balance between adding new features and fixing bugs.
- Even the smallest change was making our game buggy or breaking our core loop.
- Our project organization wasn't the best since we have a lot of scripts and game objects floating around.



Development Process Postmortem

Effective Recommendations

- Build a profound feature list before coding. This is to avoid going back and forth and having to delete past code, losing time.
- Schedule bug-fixing code and playtesting together (even more so if there is a multiplayer game).
- Keep scripts short but make it easy to make changes. Also be careful when making scripts overlap and rely on each other too much, it can make things break down easier.

Thanks !

Any questions?



CREDITS: This presentation template was created by **Slidesgo**, including icons by **Flaticon**, infographics & images by **Freepik**

